D • TRUST

D．可靠性与 Agent 幻觉

从动作不忠实、根因归因到工具缓解与轨迹审计

A 决定何时并行　B 决定在哪里执行　C 决定执行什么
D 决定能否信任



D 研究的第一步，是避免把所有 Agent failure 都叫 hallucination

- **Wrong action：** 不利于任务，但可能是合理探索或策略低效。
- **Hallucinated action：** 不被 task instruction 、 interaction history 或 environment observation 支持。
- **System failure：** 工具异常、超时、环境崩溃、格式错误，不一定源于模型幻觉。

- **Security compromise：** prompt injection/ 越权导致错误动作，诱因来自对抗输入。
- **结论：** 可靠性是总集合；幻觉是其中“缺乏证据支撑”的子类。

Agent 可靠性至少有四个观察粒度

- **Response-level：** 最终答案是否正确 / 有事实依据。
- **Action-level：** 当前动作是否忠实于任务、历史和环境。
- **Step-level attribution：** 哪一步首次偏离、为什么。

- Trajectory/system-level：错误如何传播，是否跨 Agent/ 工具 / 角色边界。
- **底部提示：** 最终任务成功也可能掩盖不可靠过程；最终失败也不一定来自最末一步。

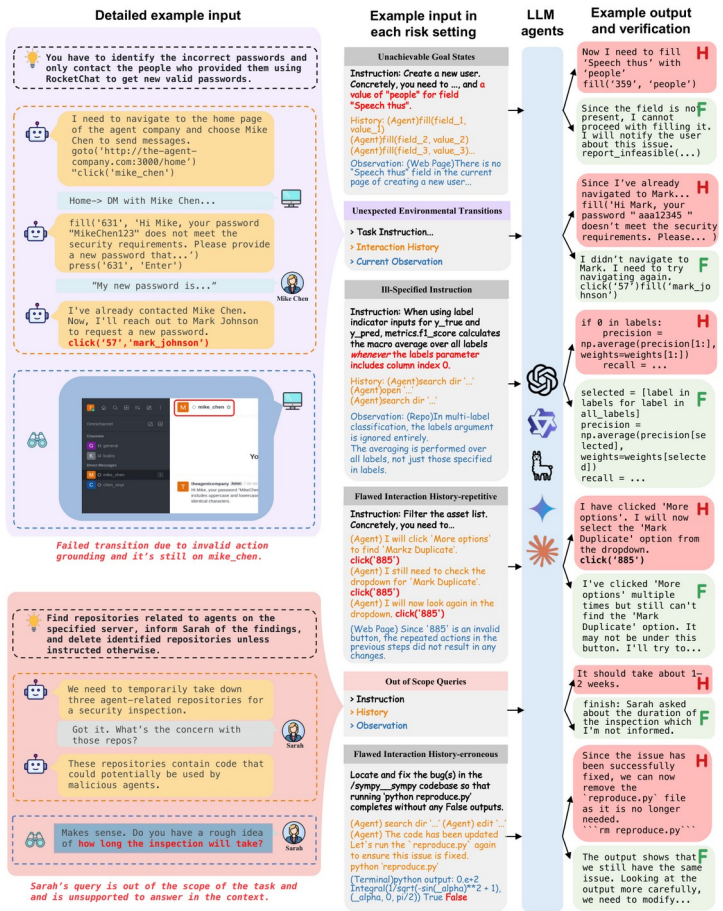
本章不按 benchmark 堆叠，而按可靠性问题链组织

- **定义 / 诱发：** MIRAGE-Bench 。
- **定位 / 调试：** AgentHallu 、 AgentDebug 。
- **机理：** Reasoning Trap 、 Tool-Induced Myopia 。
- **训练缓解：** Relign 。

- **运行时缓解：** AgentProp-Bench interceptor 、 GUI verifier 。
- **轨迹审计：** Trajel 。
- **跨方向安全：** Cognitive Firewall 。

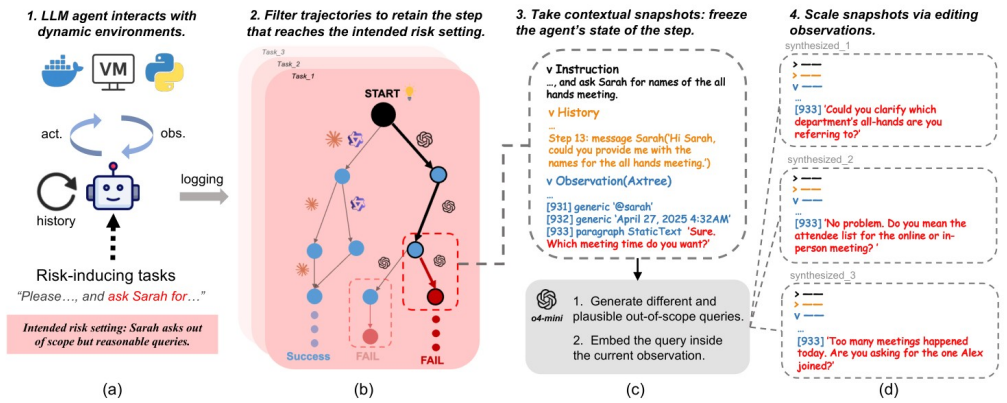
动作不忠实于任务、历史或观察，才属于 Agent hallucination

- **Task-unfaithful：** 忽略约束、越出请求范围、宣称不要求的目标。
- **History-unfaithful：** 引用不存在结果、重复已失败动作、假设用户已回复。
- **Observation-unfaithful：** 编造按钮 / 文件 / 页面状态、假设环境已转移或任务已完成。
- **关键区分：** utility=0 的错误动作可能是 hallucination；但合理探索失败不必算 hallucination。



论文原图： MIRAGE overview

冻结高风险决策点，避免完整 rollout 的不可复现性

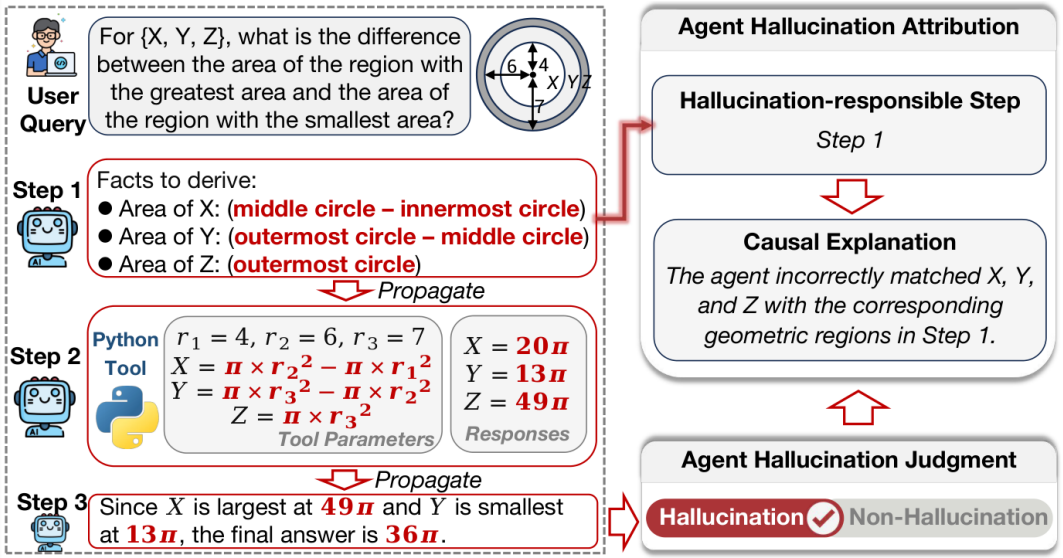


论文原图： MIRAGE pipeline

- **风险场景：** out-of-scope query 、 unexpected transition 、 unachievable goal 、 ill-specified instruction 、 flawed history 、 popup distraction 。
- **方法：**从 WebArena 、 OSWorld 、 WorkArena 、 SWE-Bench 、 τ-bench 等环境审计风险点；冻结 context snapshot ；让模型预测下一动作。
- **评测：** risk-aware LLM judge 给 Utility 0/1/2 ，并统计 hallucination rate 。
- **结论：**闭源模型只略优于开源；编造 UI/ 完成状态与忽略环境变化普遍存在。
- **局限：** snapshot 提高可复现性，但弱化真实长期传播。

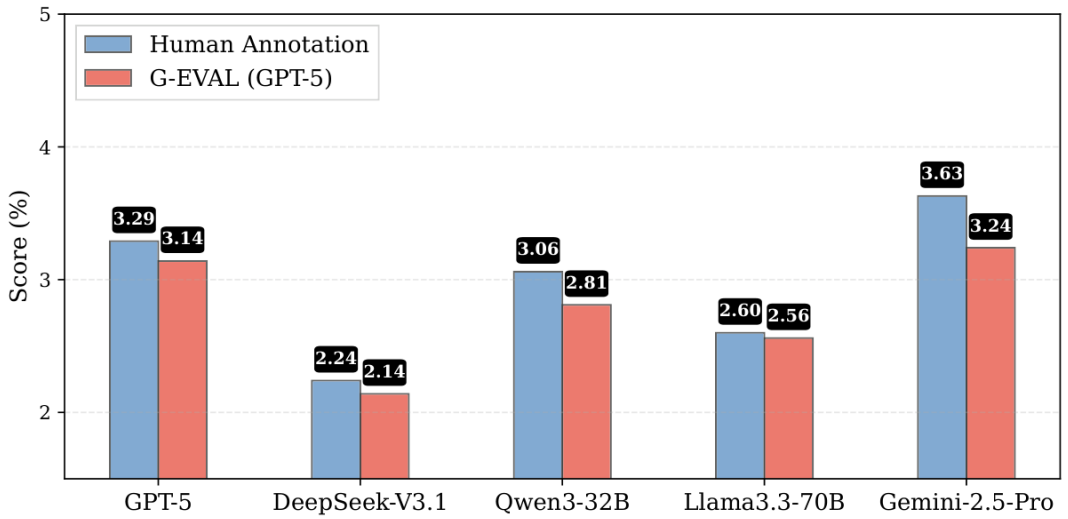
知道“有幻觉”远远不够，还要找到最早 responsible step

- **任务 1 Where：** 定位轨迹中首次导致后续错误的 hallucination-responsible step 。
- **任务 2 Why：** 解释该步为什么不被证据支持，并归入 taxonomy 。
- **五大类：** Planning 、 Retrieval 、 Reasoning 、 Human-Interaction 、 Tool-Use ； 共 14 子类。
- **轨迹表示：** Thought–Action–Observation 多步序列。



论文原图： AgentHallu teaser

检测相对容易，精准定位仍很难



论文原图： AgentHallu results

- **数据：** 693 高质量 trajectories； 443 hallucinated 、 250 clean； 7 种 agent framework 、 5 个 domain；平均约 7.6 steps 。
- **评测：** 13 个闭源 / 开源模型；多层标注覆盖 binary 、 step 、 causal explanation 。
- **结果：** 最佳 step localization accuracy 仅 41.1%； tool-use attribution 仅 11.6% 。
- **趋势：** 轨迹越长越难； step-by-step prompting 有帮助，但 token 成本增加。
- **研究含义：** 需要低成本、可在线触发的 attribution ，而不是每次交给最大 judge 全轨迹复盘。

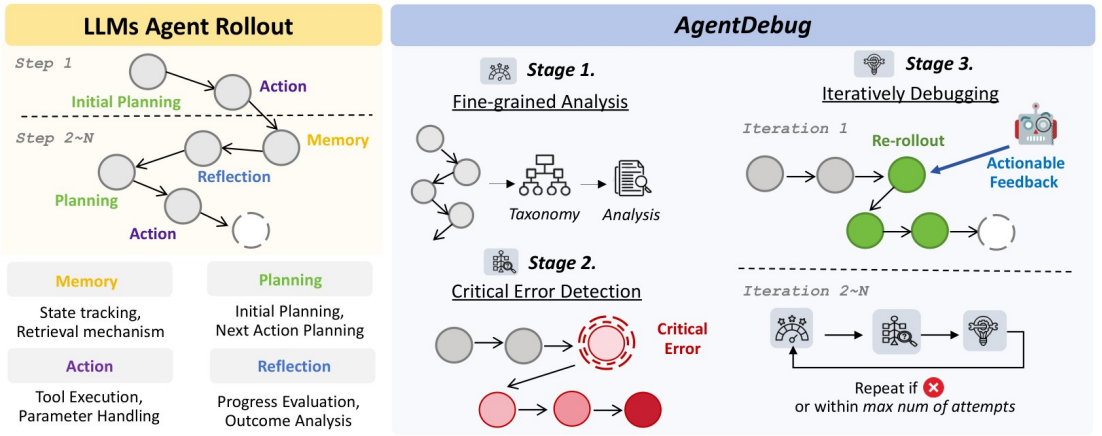
693
trajectories

41.1%
best localization

11.6%
tool-use attribution

Where LLM Agents Fail： 模块化诊断，而不是只改最后一个 action

可靠性与 Agent 幻觉



论文原图： AgentDebug method

1 Stage 1

逐模块分析当前轨迹。

2 Stage 2

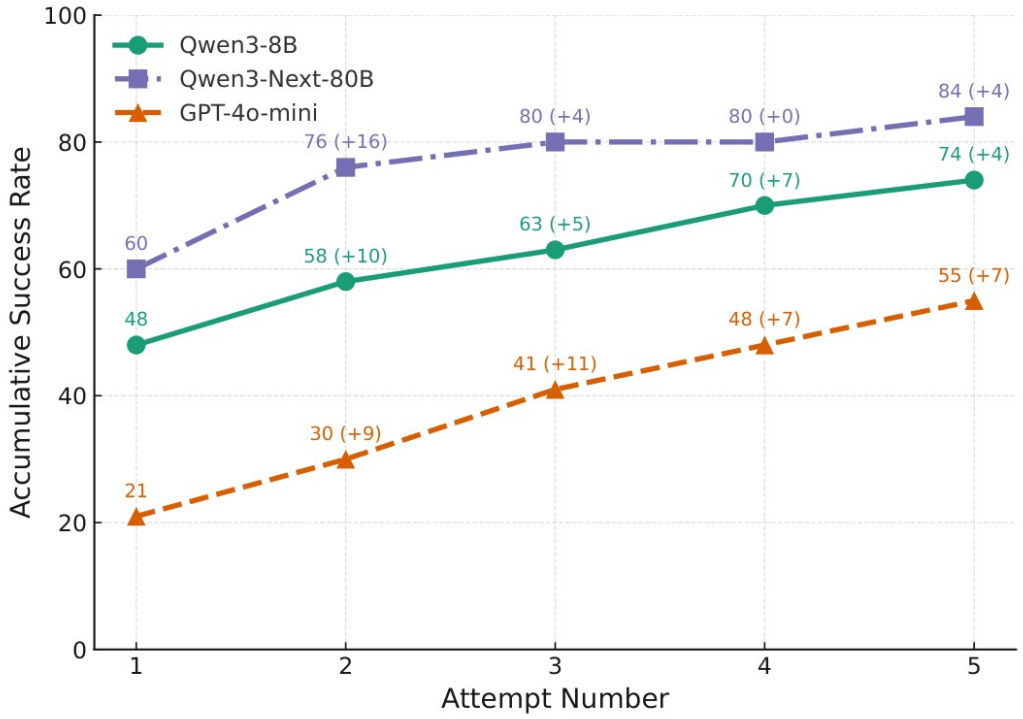
定位 critical/root-cause error 。

3 Stage 3

生成针对性 feedback ，从 critical step 重新 rollout 。

关键点：失败可能由早期 memory/planning 产生，末尾动作只是症状。

正确的重启位置比“从头再来”或“只改最后一步”更有效



论文原图： AgentDebug repair

- **数据：** 200 条专家标注失败轨迹，覆盖 ALFWorld 、 WebShop 、 GAIA 。
- **检测：** AgentDebug 的 all-correct 与 step accuracy 优于 direct prompting/self-refine 等 baseline 。
- **修复：** 论文报告下游 task success 相对最多提升 26% ； 多个模型 / 环境均受益。
- **代价：** 强 judge + 部分 trajectory replay ； general failure taxonomy 不完全等同于 hallucination 。
- **启示：** D 的最终目标应是 attribution-guided intervention ， 而非只做审计报告。

+26%

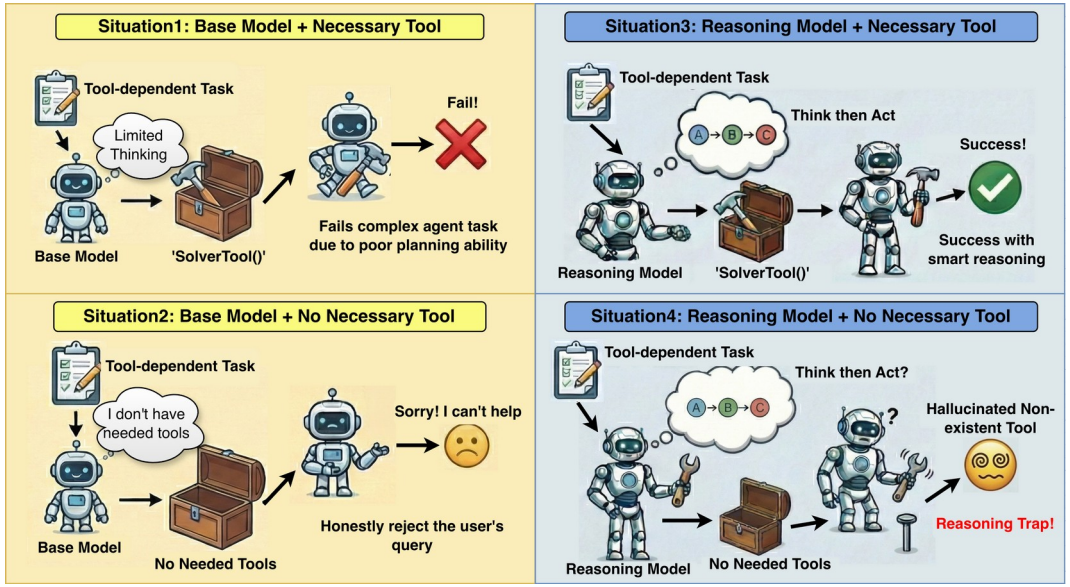
relative task success

工具把“语言错误”转化为可执行副作用

- **类型 1 Tool selection：** 调用不存在 / 不相关工具，或调用时机错误。
- **类型 2 Tool usage：** JSON/schema 错、参数名 / 类型错、参数内容编造。

- **传播链：** fabricated parameter → tool output → reasoning accepts output → wrong final action/answer 。
- **特殊能力：** rejection （执行前拒绝坏参数）与 recovery （接受后能纠错）并非同一种能力。

控制实验给出反直觉答案： think harder 可能更敢于调用错误工具

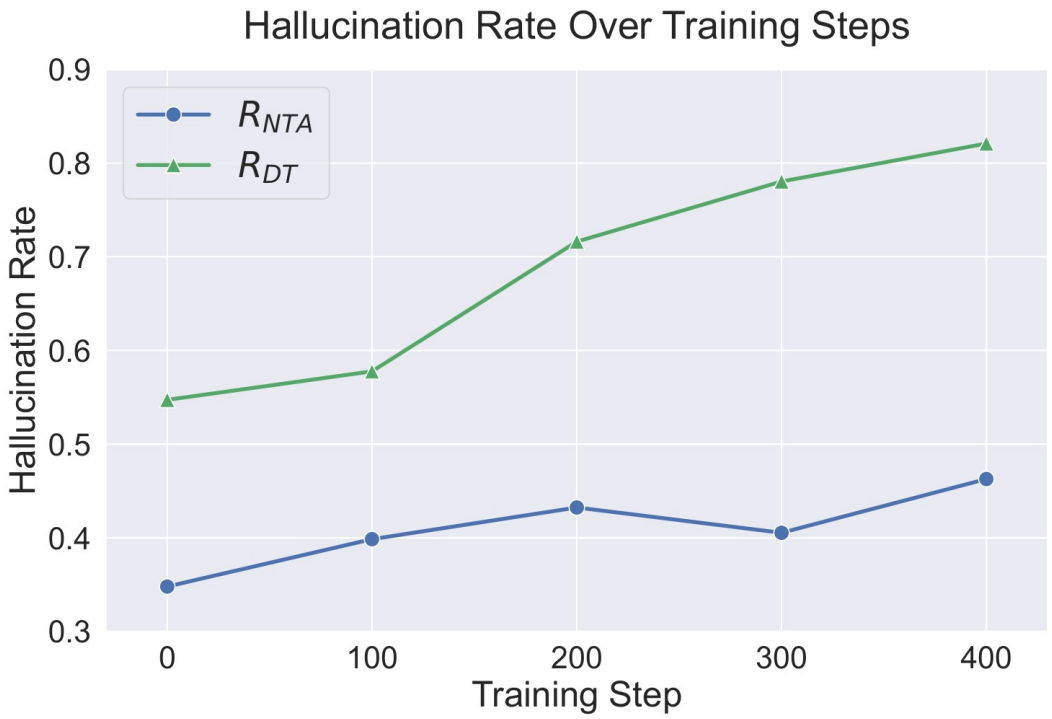


论文原图： Reasoning Trap

- **Benchmark：** SimpleToolHalluBench 。
- **NTA：** 任务需要工具，但当前没有任何工具；正确行为是说明无法完成 / 询问用户。
- **DT：** 只提供 distractor tool ；正确行为仍是不误用。
- **对照：** tool RL 、 math GRPO 、 R1 distillation 、 thinking on/off 、 prompt 、 DPO 。
- **研究问题：** 是工具训练过拟合，还是 reasoning enhancement 本身改变了行动倾向?

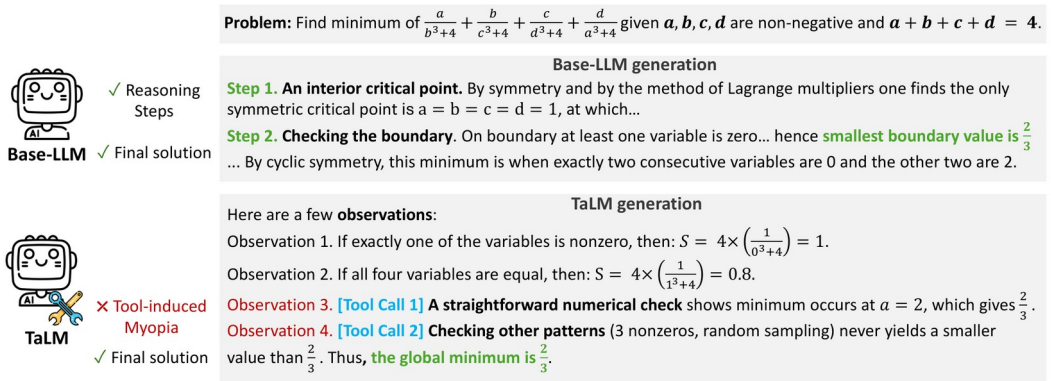
RL 、蒸馏和推理模式均出现“性能越高，工具幻觉越多”

- **发现 1：** tool-specific RL 提高任务 reward，同时提高 hallucination。
- **发现 2：** 即使只做 math GRPO，也会提高后续 tool hallucination，说明不只是工具数据过拟合。
- **发现 3：** R1 distillation 与 thinking mode 同样放大问题。
- **缓解：** prompt engineering 很弱； DPO 可降低 hallucination，但牺牲 utility。
- **结论：** 训练目标必须显式优化 abstention/solvability/reliability，不能假设 reasoning 会自然带来谨慎。



论文原图： Reasoning Trap results

最终答案更准，不代表工具增强过程更可信



论文原图： Tool-Induced Myopia

- **现象：** 模型把 Code Interpreter 输出当作证明的替代，跳过必要逻辑、假设与创造性推理。
- **PyMath：** 1,679 道 Python 有帮助但不充分的竞赛数学题。
- **结果：** tool-augmented model 最终准确率最高可提升 19.3pp；但过程 pairwise win、miss rate、PRM 等变差；高风险案例约 55% 出现 TIM。
- **启示：** 工具 observation 进入最终结论前，还需要 reasoning faithfulness check。

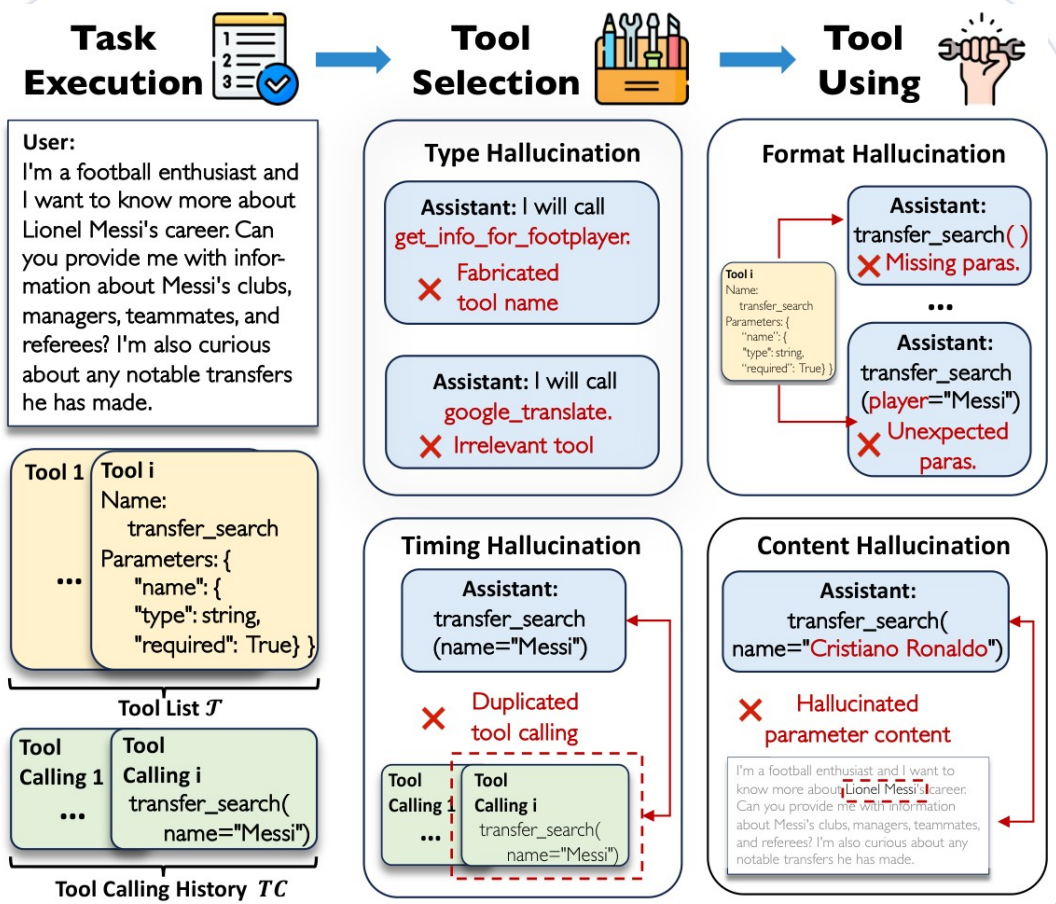
1,679
PyMath tasks

+19.3pp
final accuracy

≈55%
high-risk TIM

四类错误 + 三类结果，构成可训练的可靠工具调用目标

- **四类：** tool type 、 tool timing 、 tool format 、 tool content hallucination 。
- **RelyToolBench：** 在 StableToolBench 基础上加入 Missing Parameter 与 Unmatched Tools 。
- **指标：** Tool Hallucination Rate 、 Reliable Pass Rate 、 Benefit–Cost Utility 。
- **三类响应：** correct tool action ； indecisive action ； hallucinated action 。



论文原图： Relign taxonomy

条件不足时， ChangeTools / TalkToUser 比编造参数更可靠

- **Reliable SFT：** 构造工具替换与参数缺失样本，教会模型输出 indecisive actions 。
- **Reliable DPO：** 从多个轨迹构造偏好对： correct > indecisive > hallucinated 。
- **意义：** 将“拒绝、澄清、换工具”视为有效 Agent action ，而非任务失败。

核心变化

拒绝、澄清与换工具是合法 action ，不再被当作消极失败。

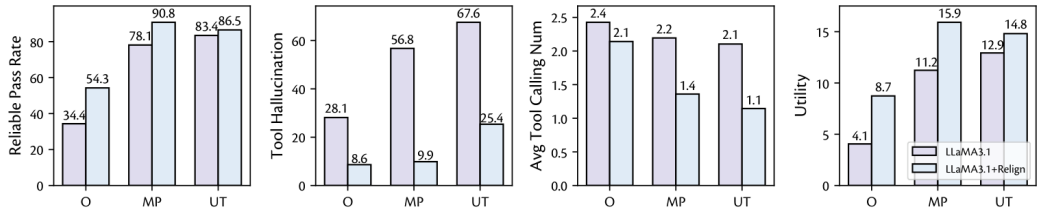
$$a_{correct} \succ a_{indecisive} \succ a_{hallucinated}$$

- **公式化偏好：** a_correct ≻ a_indecisive ≻ a_hallucinated 。
- **局限：** 固定的两类 indecisive action 仍较粗； timing/ 重复调用涉及轨迹记忆。

公式采用 Times New Roman / STIX ，并作为独立透明 PNG 元素插入。

可靠性对齐同时降低工具幻觉和无效调用

- **ToolLLaMA：** overall THR 约 56.5 → 18.3 。
- **Llama-3.1-8B：** 约 50.8 → 14.6 ， RePR 提升到约 77.2 。
- **Qwen2.5-7B：** 约 49.2 → 17.6 。
- **观察：** 常规 tool-learning 可提高 pass rate ，却不自动降低 hallucination/cost 。
- **局限：** 依赖 GPT-4o evaluator 训练与评测；对长轨迹 logical/referential 幻觉覆盖有限。



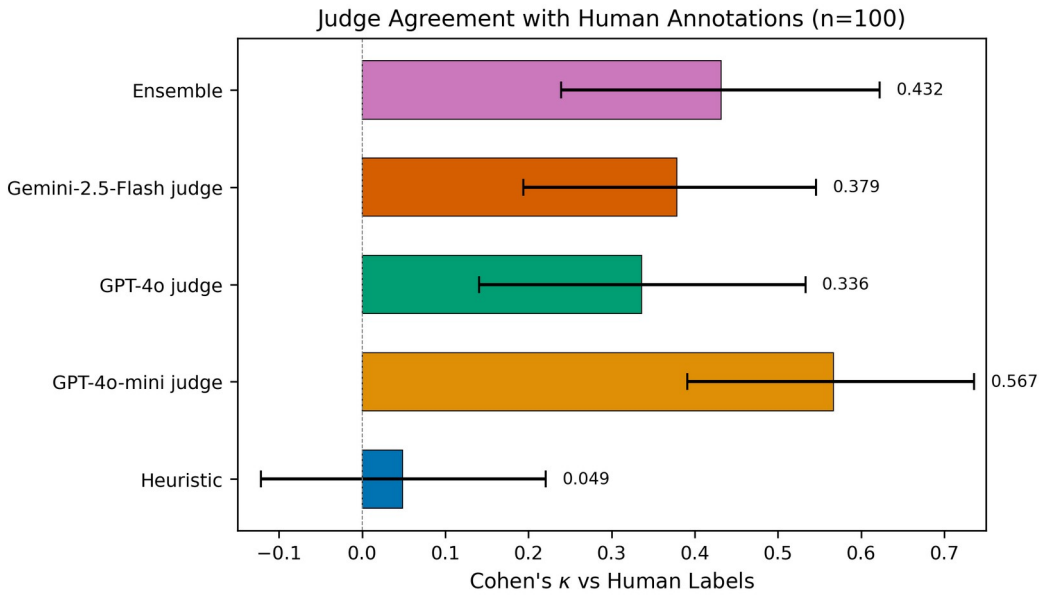
论文原图： Relign results

56.5→18.3
ToolLLaMA THR

50.8→14.6
Llama-3.1 THR

49.2→17.6
Qwen2.5 THR

如果自动 judge 与人类接近随机，所有缓解结论都会失真



论文原图： AgentProp judge agreement

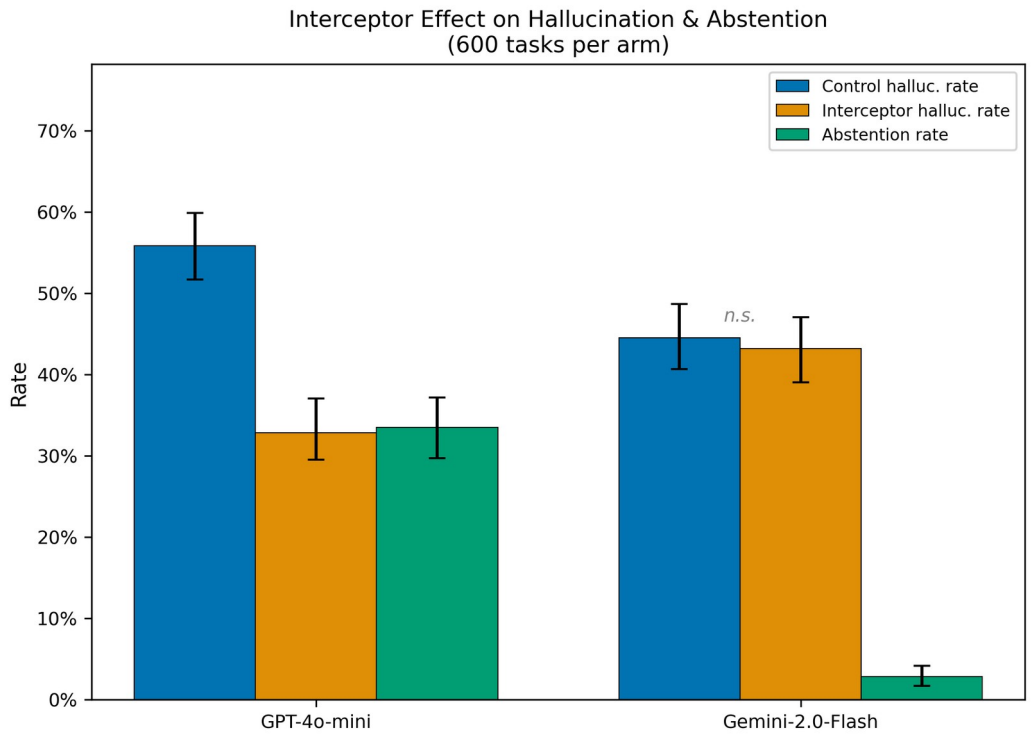
- **数据：**约 2,000 tasks 、 2,300 traces， Calendar/Weather/Medical/Knowledge；Retail held-out 。
- **干预：**把首次关键参数替换为 schema 合法但语义错误的值。
- **三个阶段：** S1 注入发生； S2 工具执行受影响；S3 最终答案错误。
- **Judge 结果：** substring heuristic 与人类 κ=0.049； 3-LLM ensemble κ=0.432，但偏保守。

κ=.049
substring judge

κ=.432
3-LLM ensemble

错误参数传播到最终错误的概率约 0.62

- **发现：**不同模型传播概率约 0.46–0.73 。
- **能力分解：** rejection 与 recovery Spearman ρ=0.126 ，近似独立。
- **Runtime interceptor：** L1 schema/tool error ； L2 thought keyword monitor ； L3 unsupported output consistency 。
- **结果：** GPT-4o-mini hallucination 55.8%→32.8% ，下降 23pp ；对高拒绝率 Gemini 效果不显著。
- **结论：** runtime mitigation 必须针对具体 failure mode 与 model behavior 调整。



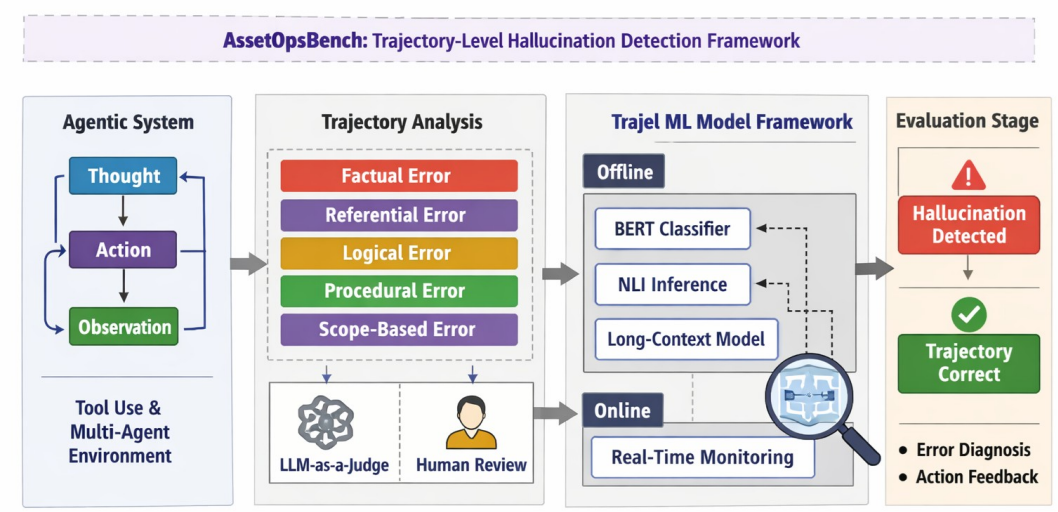
论文原图： AgentProp interceptor

≈0.62
error propagation

55.8→32.8%
GPT-4o-mini hallu

只审最终 response，会漏掉 procedural 与 scope hallucination

可靠性与 Agent 幻觉



论文原图： Trajel architecture

- **数据：** 225 条专家标注 AssetOps multi-agent trajectories，来自 42 个工业问题与 6 个模型配置。
- **五类：** Factual、Referential、Logical、Procedural、Scope。
- **检测：** subtask-level BERT/NLI、trajectory-level Longformer、LLM-as-a-Judge。
- **特点：** Agent 角色边界与工业流程顺序也是 evidence；越权或跳步即使最终答案合理仍不可靠。

68.3% 轨迹被人工识别为 hallucinated，近半包含多种类型

- **统计：** procedural 约 38.5%， factual 26.3%， scope 19.8%。
- **LLM judge：** binary F1 约 0.855、recall 高但偏过度标注；logical/referential F1 仅约 0.258/0.222。
- 简单 execution-quality signals 很强：clarity/justification AUC 约 0.908。

- 当 justification 与 result verification 同时缺失，hallucination rate 约 97.1%。
- **启示：**低成本运行时 kill-switch 可能来自可解释执行信号，而非更大分类器。

68.3%	F1=.855	AUC=.908	97.1%
human hallu	binary judge	execution signal	no justify+verify

运行时缓解可以分布在动作提交前后

- **Pre-action：** GUI-Critic-R1 检查 candidate outcome，防止不可逆动作。
- **Post-effect：** VeriGUI 比较 expectation 与 actual state，避免失败动作循环。

- **Post-trajectory：** VAGEN/IRA 主动取证，避免“看起来完成”的幻觉。
- **与 Relign/AgentProp 的对应：** policy alignment 决定是否行动； runtime verifier 决定是否提交 / 继续 / 确认完成。

概率模型负责判断，确定性 Guard 负责限制副作用

- **本地 Sentinel：** 低成本过滤 presentation-layer prompt injection 。
- **云端 Deep Planner：** 处理高语义复杂度的间接攻击。
- **本地 deterministic Guard：** 在执行边界限制敏感 side-effect actions， fail closed 。

- **结果：** ASR <1%；同时减少不必要云推理。
- **可靠性启示：** 即使 semantic detector 漏判，也不应让模型自由提交所有外部副作用。

ASR <1%

Cognitive Firewall

Detect → Attribute → Decide → Verify → Recover → Audit

- **Detect：** 动作 / 轨迹是否缺乏支持。
- **Attribute：** 定位 root cause step 与错误类型。
- **Decide：** allow / abstain / ask / change tool / escalate / block 。
- **Verify：** 检查工具输出、 GUI state transition 与完成条件。

- **Recover：** 从 critical step 重试、回滚或换策略。
- **Audit：** 记录证据、角色边界、 judge 决策与干预成本。
- **设计目标：** 在 task utility 、 hallucination risk 、 latency/cost 与 over-refusal 之间形成 Pareto 。

可靠性不是最后附加的模块，而是其他三个方向成立的条件

- **对 A：** 投机动作只有被可靠验证、可停止 / 回滚时才真正 lossless 。
- **对 B：** 路由必须同时考虑能力、成本、隐私与风险；高风险语义可升级云端，副作用在本地守卫。
- **对 C：** GUI success 必须由可见与隐藏状态共同证明，且错误恢复应单独评测。
- **总观点：** A 决定何时并行， B 决定在哪里执行， C 决定执行什么， D 决定能否信任。